Logistic Regression Example - Problem 10.1 Financial Condition of Banks

> (c) 2025 Galit Shmueli, Peter C. Bruce, Peter Gedeck
>
> Code included in
>
> *Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python* (Second Edition) Galit Shmueli, Peter C. Bruce, Peter Gedeck, and Nitin R. Patel. 2025.

Add comments to each **code** cell (no comments need to be added for Markdown).

Each code cell starts with two comments:

1. Your initials and last two digits of your ID number.
2. Provide a brief explanation of the code cell. If comments are already included in the code cell, you may revise them for clarity, summarize the existing comments, or leave them unchanged if they are already appropriate. For example,

JT88

This code cell imports necessary Python packages.

The dataset Banks.csv includes data on a sample of 20 banks. The "Financial Condition" column records the judgment of an expert on the financial condition of each bank. This outcome variable takes one of two possible values -- weak or strong-- according to the financial condition of the bank. The predictors are two ratios used in the financial analysis of banks: TotLns&Lses/Assets is the ratio of total loans and leases to total assets, and TotExp/Assets is the ratio of total expenses to total assets. The target is to use the two ratios for classifying the financial condition of a new bank.

Run a logistic regression model (on the entire dataset) that model the status of a bank as a function of the two financial measures provided. The Financial Condition column encodes weak as 1 and strong as 0. Use the default cutoff of 0.5.

In [1]:
```
import numpy as np
import pandas as pd
import math # import Python's math module with mathematical functions that a
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression, LogisticRegressionCV
from sklearn.metrics import accuracy_score
from mlxtend.feature_selection import SequentialFeatureSelector
import matplotlib.pylab as plt
import mlba
```

```
no display found. Using non-interactive Agg backend
```

10.1.a. Write the estimated equation that associates the financial condition of a bank with its two predictors in three formats: The logit as a function of the predictors; The odds as a function of the predictors; The probability as a function of the predictors

In [2]:
```
#VB67
#loading the dataset
bank_df = mlba.load_data('banks.csv')
#displaying the first five rows of the dataset
bank_df.head()
```

Out[2]:

	Obs	Financial Condition	TotCap/Assets	TotExp/Assets	TotLns&Lses/Assets
0	1	1	9.7	0.12	0.65
1	2	1	1.0	0.11	0.62
2	3	1	6.9	0.09	1.02
3	4	1	5.8	0.10	0.67
4	5	1	4.3	0.11	0.69

In [3]:
```
#VB67
# renaming is practically necessary here as special characters & / and space
bank_df = bank_df.rename(columns = {
    'Financial Condition': 'Financial_Condition',
    'TotExp/Assets': 'TotExp_Assets',
    'TotLns&Lses/Assets': 'TotLnsLses_Assets',
})
#removing the mentioned column which are not relevant
bank_df = bank_df.drop(columns = ['Obs','TotCap/Assets'])
#displaying the first five rows of the column
bank_df.head()
```

Out[3]:

	Financial_Condition	TotExp_Assets	TotLnsLses_Assets
0	1	0.12	0.65
1	1	0.11	0.62
2	1	0.09	1.02
3	1	0.10	0.67
4	1	0.11	0.69

In [9]:
```
#VB67
#set penalty=l2 and C=1e42 to avoid regularization
logit_reg = LogisticRegression(penalty="l2", C=1e42, solver='liblinear')
y = bank_df['Financial_Condition']
X = bank_df.drop(columns=['Financial_Condition'])
logit_reg.fit(X, y)
```

Out[9]:

```
LogisticRegression
LogisticRegression(C=1e+42, solver='liblinear')
```

In [10]:

```
#VB67
print(f'intercept {logit_reg.intercept_[0]:.6f}')
#Coefficients for each predictor
print(pd.DataFrame({'coeff': logit_reg.coef_[0]}, index=X.columns).transpose
#AIC score for model evaluation
print(f'AIC {mlba.AIC_score(y_true=y, y_pred=logit_reg.predict(X), df = len(
```

```
intercept -14.720833
       TotExp_Assets  TotLnsLses_Assets
coeff      89.832567           8.371267
AIC 18.706
```

Answer:

The logit as a function of the predictors:

logit = -14.7208 + (89.8326 * TotExp_Assets) + (8.3713 * TotLnsLses_Assets)

The odds as a function of the predictors:

Odds = e^(logit) = e^(-14.7208 + (89.8326 * TotExp_Assets) + (8.3713 * TotLnsLses_Assets))

The probability as a function of the predictors:

p = (1 + Exp[-(-14.7208 + (89.8326 * TotExp_Assets) + (8.3713 * TotLnsLses_Assets))])^-1

10.1.b. Consider a new bank whose total loans and leases/assets ratio = 0.6 and total expenses/assets ratio = 0.11. From your logistic regression model, estimate the following four quantities for this bank (use Python to do all the intermediate calculations; show your final answers to four decimal places): the logit, the odds, the probability of being financially weak, and the classification of the bank (use cutoff = 0.5).

In [12]:

```
#VB67
# Creating a new observation (bank) with specific values for predictors:
new_bank = pd.DataFrame(
    [[0.11, 0.6]],
    columns=['TotExp_Assets', 'TotLnsLses_Assets'])
# Displaying the new observation
new_bank
```

Out[12]:

	TotExp_Assets	TotLnsLses_Assets
0	0.11	0.6

In [13]:

```
#VB67
# Compute logit for the new bank manually using the model coefficients
logit_new = -14.7210 + (89.8339 * new_bank.TotExp_Assets[0]) + (8.3713 * new
# Converting logit to odds
odds = math.exp(logit_new)
```

```
# Converting odds to probability
prob = odds/(1+odds)

# Displaying manual calculations
print(f'logit = {logit_new:.4f}')
print(f'odds = {odds:.4f}')
print(f'prob = {prob:.4f}')

# Model predictions using the fitted logistic regression
print('\nModel predictions:')
print(f'class: {logit_reg.predict(new_bank)[0]}')
print(f'probability : {logit_reg.predict_proba(new_bank)[0][1]:.4f}')
```

```
logit = 0.1835
odds = 1.2014
prob = 0.5457

Model predictions:
class: 1
probability : 0.5457
```

Probability that the new bank is 0.5457 and therefore the predicted class for this new bank is 1, or "financially week".

10.1.c. The cutoff value of 0.5 is used in conjunction with the probability of being financially weak. Compute the threshold that should be used if we want to make a classification based on the odds of being financially weak, and the threshold for the corresponding logit.

Cutoff value of p=0.5.

Odds = (p) / (1-p) = (0.5) / (1-0.5) = 1

If odds > 1 then classify financial status as "weak" (otherwise classify as "strong").

Logit = ln (odds) = ln (1) = 0

If Logit > 0 then classify financial status as "weak" (otherwise, classify it as "strong")

Therefore, a cutoff of 0.5 on the probability of being weak is equivalent to a threshold of 1 on the odds of being weak, and to a threshold of 0 on the logit.

10.1.d. Interpret the estimated coefficient for the total loans & leases to total assets ratio (TotLns&Lses/Assets) in terms of the odds of being financially weak.

VB67

A positive coefficient in the logit model translates into a coefficient larger than 1 in the odds model. In the logit model, the estimated coefficient for the total loans & leases-to-assets ratio is 8.37. In the odds models, the coefficient is e^8.37 = 4316. This means

that an increase of a unit in total loans and leases-to-assets ratio is associated with an increase in the odds of being financially weak by a factor of 4316, holding all other variables constant.

10.1.e. When a bank that is in poor financial condition is misclassified as financially strong, the misclassification cost is much higher than when a financially strong bank is misclassified as weak. To minimize the expected cost of misclassification, should the cutoff value for classification (which is currently at 0.5) be increased or decreased?

VB67

In order to minimize the expected cost of misclassification in this case, we need to decrease the cutoff. When the cost of misclassifying a financially weak bank as strong is significantly higher than misclassifying a strong bank as weak, the classification threshold (cutoff) should be lowered from the default of 0.5. By decreasing the cutoff, the model becomes more sensitive to identifying weak banks—it classifies a bank as weak even when the predicted probability is moderately low. This increases the likelihood of correctly identifying banks in poor financial condition, thereby reducing the risk of high-cost false negatives. Although this approach may increase the number of false positives (strong banks misclassified as weak), the trade-off is justified because the cost associated with failing to detect a weak bank is much greater.

In [ ]:

```
```